

00 — Increasing atmospheric CO₂ and ocean acidification

Exercise 9: answer sheet (a–h). Use PyCO2SYS to calculate carbonate-system properties from two inputs and interpret changes in pH and mineral saturation. Write your calculations and short answers below each question. The single worked example uses different inputs from the exercise; use the documentation to adapt it.

These are separate **equilibrium chemistry calculations**, not a time-dependent ocean model. The CO₂ values are the prescribed values in the exercise, not live observations. Coupled ocean–atmosphere experiments follow in notebook 04.

Reading key: **Key term** = concept to notice; **Question** = student prompt. Instructor answers use labelled purple panels in the instructor sheet.

LECTURER REVIEW | INSTRUCTOR EDITION

Executed in ESBMTK314. All instructor solutions, code, execution outputs and optional disclosures are included. No Jupyter installation is needed to read this PDF.

Source: 00_PyCO2SYS.ipynb

Execution: 2026-09-24. The source notebooks are unchanged.

Conditions and documentation

Use **TA = 2100 μmol/kg**, **T = 15 °C**, **salinity = 35** and **seawater pressure = 0 dbar**, unless a question changes one of them. Return to these **baseline conditions** for each new part. In temperature/salinity comparisons, hold TA and atmospheric CO₂ fixed and allow DIC to adjust to equilibrium. Do not use `temperature_out` for these comparisons: it describes a different constraint on the water sample.

Keep `config.chemistry` **unchanged**: `opt_k_carbonic=10`, `opt_pH_scale=2` (**seawater scale**), `opt_buffers_mode=1`. Only the exercise temperatures differ from the shared 16 °C setting used in 01/02. Nutrient totals are zero, atmospheric pressure is 1 atm, and other PyCO2SYS options retain their defaults.

The exercise uses “ppm (or μatm)” approximately. Follow the existing notebook's convention: enter atmospheric values as **dry-air xCO₂ in ppm** (type 9). PyCO2SYS accounts for water vapour when relating xCO₂ to pCO₂; pCO₂ in μatm (type 4) is a different input. Small differences from the supplied answer key are expected; do not change the constant options to match its rounded numbers.

See [How to use PyCO2SYS: input pairs, settings and results](#), especially [carbonate-system parameter types](#). Concentrations are in μmol/kg of seawater; pH and saturation states are dimensionless.

```
from pathlib import Path
import sys

ROOT = next(p for p in (Path.cwd(), *Path.cwd().parents)
             if (p / 'teaching_config.py').is_file())
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import PyCO2SYS as pyco2
from teaching_config import TEACHING as config

TEMPERATURE_C = 15.0
SALINITY = 35.0
PRESSURE_DBAR = 0.0
INITIAL_TA = 2100.0 # umol/kg
```

```
print('PyCO2SYS version:', pyco2.__version__)
print('Carbonate settings:', config.chemistry)
```

PyCO2SYS version: 1.8.3.4

Carbonate settings: {'opt_k_carbonic': 10, 'opt_pH_scale': 2, 'opt_buffers_mode': 1}

One worked example: TA and DIC as inputs

For a separate sample, prescribe TA = 2300 and DIC = 2000 $\mu\text{mol/kg}$. `par1_type=1` means TA and `par2_type=2` means DIC. The result is a dictionary: select the output you need by its key. Adapt this call to the input pair and conditions required by the exercise.

```
example = pyco2.sys(
    par1=2300.0, par1_type=1, # TA, umol/kg
    par2=2000.0, par2_type=2, # DIC, umol/kg
    temperature=TEMPERATURE_C,
    salinity=SALINITY,
    pressure=PRESSURE_DBAR,
    **config.chemistry,
)
print(f"Seawater-scale pH: {float(example['pH']):.3f}")
print(f"DIC: {float(example['dic']):.1f} umol/kg")
print(f"Aragonite saturation: {float(example['saturation_aragonite']):.3f}")
print(f"Calcite saturation: {float(example['saturation_calcite']):.3f}")
```

Seawater-scale pH: 8.190

DIC: 2000.0 $\mu\text{mol/kg}$

Aragonite saturation: 3.203

Calcite saturation: 4.987

Instructor reference

Instructor reference

All answers below use PyCO2SYS 1.8.3.4 and the unmodified `config.chemistry` choices. Rounded values are included so this sheet can be read without running it; the cells reproduce the calculations. The supplied helper only avoids repeating the same equilibrium call. It and all solutions are removed from the student copy.

```
def equilibrium(xco2, temperature=TEMPERATURE_C, salinity=SALINITY):
    return pyco2.sys(
        par1=INITIAL_TA, par1_type=1,
        par2=xco2, par2_type=9,
        temperature=temperature, salinity=salinity,
        pressure=PRESSURE_DBAR, **config.chemistry,
    )

def show_state(label, result):
    print(f"{label}: pH = {float(result['pH']):.3f}; "
          f"DIC = {float(result['dic']):.1f} umol/kg; "
          f"TA = {float(result['alkalinity']):.1f} umol/kg; "
          f"Omega_aragonite = {float(result['saturation_aragonite']):.3f}; "
          f"Omega_calcite = {float(result['saturation_calcite']):.3f}")
```

a) Present-day pH

Question — part a

Take the exercise's present-day atmospheric CO₂ value of **430 ppm**. Calculate the pH on the seawater scale at the baseline conditions.

```
# Your calculation
```

```
present = equilibrium(430.0)
show_state('Present', present)
```

Present: pH = 7.982; DIC = 1918.7 umol/kg; TA = 2100.0 umol/kg; Omega_aragonite = 1.975; Omega_calcite = 3.075

Instructor answer

The seawater-scale pH is **7.982** (about 7.98). TA and xCO₂ are prescribed; pH and DIC are calculated. This represents water equilibrated with the specified atmosphere at the chosen conditions.

b) Temperature sensitivity

Question — part b

How would pH change if the ocean temperature were **5 °C** or **25 °C**? Keep TA = 2100 μmol/kg, salinity = 35 and xCO₂ = 430 ppm. Compare with 15 °C.

```
# Your calculation
```

```
temperature_states = {}
for temperature in (5.0, 15.0, 25.0):
    state = equilibrium(430.0, temperature=temperature)
    temperature_states[temperature] = state
    print(f"T = {temperature:g} degC: pH = {float(state['pH']):.3f}; "
          f"change from 15 degC = {float(state['pH'] - present['pH']):+.3f}")
```

T = 5 degC: pH = 7.971; change from 15 degC = -0.011

T = 15 degC: pH = 7.982; change from 15 degC = +0.000

T = 25 degC: pH = 7.985; change from 15 degC = +0.003

Instructor answer

Temperature (°C)	pH	Change from 15 °C
5	7.971	-0.011
15	7.982	0.000
25	7.985	+0.003

The direct pH effect is small **at fixed TA and atmospheric xCO₂**. Changes in CO₂ solubility and acid–base equilibria partly offset one another while DIC adjusts. This does not imply that warming a closed sample at fixed DIC has the same effect, or that temperature has little effect on mineral saturation or biology.

c) Salinity sensitivity

Question — part c

Calculate pH at salinity **38** (a Mediterranean-like value) and **32** (a fresher value). Keep T = 15 °C, TA = 2100 µmol/kg and xCO₂ = 430 ppm. How large is the direct effect compared with salinity 35?

Your calculation

```
salinity_states = {}
for salinity in (38.0, 35.0, 32.0):
    state = equilibrium(430.0, salinity=salinity)
    salinity_states[salinity] = state
    print(f"S = {salinity:g}: pH = {float(state['pH']):.3f}; "
          f"change from S = 35: {float(state['pH'] - present['pH']):+.3f}")
```

S = 38: pH = 7.975; change from S = 35: -0.007

S = 35: pH = 7.982; change from S = 35: +0.000

S = 32: pH = 7.991; change from S = 35: +0.009

Instructor answer

At salinity **38**, pH is **7.975** (−0.007); at salinity **32**, pH is **7.991** (+0.009). These are small direct changes at fixed TA and xCO₂. Actual freshwater dilution or evaporation also changes TA and other solutes; this fixed-TA comparison isolates salinity and does not simulate those processes.

d) Glacial and interglacial CO₂

Question — part d

Calculate pH for a glacial atmosphere with **180 ppm** CO₂ and an interglacial atmosphere with **270 ppm** CO₂. Would temperature differences substantially alter the comparison? Use 15 °C for both first, then use the 5–25 °C range from (b) as a sensitivity check; it is not a reconstructed climate history.

```
# Your calculation
```

```
climate_states = {}
for xco2 in (180.0, 270.0):
    for temperature in (5.0, 15.0, 25.0):
        state = equilibrium(xco2, temperature=temperature)
        climate_states[xco2, temperature] = state
        print(f"xCO2 = {xco2:g} ppm, T = {temperature:g} degC: "
              f"pH = {float(state['pH']):.3f}")
```

xCO2 = 180 ppm, T = 5 degC: pH = 8.299

xCO2 = 180 ppm, T = 15 degC: pH = 8.294

xCO2 = 180 ppm, T = 25 degC: pH = 8.281

xCO2 = 270 ppm, T = 5 degC: pH = 8.150

xCO2 = 270 ppm, T = 15 degC: pH = 8.153

xCO2 = 270 ppm, T = 25 degC: pH = 8.147

Instructor answer

Prescribed CO ₂ (ppm)	pH at 5 °C	pH at 15 °C	pH at 25 °C
180 (glacial)	8.299	8.294	8.281
270 (interglacial)	8.150	8.153	8.147

At 15 °C, glacial pH exceeds interglacial pH by **0.142**. Temperature sensitivity over this deliberately broad range is much smaller than the CO₂ effect and does not reverse it. For example, 180 ppm at 5 °C versus 270 ppm at 15 °C gives a difference of 0.146. These conditional chemistry comparisons leave changes in real-ocean circulation, salinity and alkalinity outside the calculation.

e) Change since preindustrial times and 2000

Question — part e

The exercise gives **278 ppm** for preindustrial CO₂ and **366 ppm** for the year 2000. Calculate pH at both values. How much has pH declined from preindustrial times to 2000, and from 2000 to the exercise's present (430 ppm)? Also report the total decline since preindustrial times.

```
# Your calculation
```

```
preindustrial = equilibrium(278.0)
year_2000 = equilibrium(366.0)
for label, state in [('Preindustrial', preindustrial), ('2000', year_2000),
                    ('Present', present)]:
    print(f"{label}: pH = {float(state['pH']):.3f}")
for label, earlier, later in [
    ('Preindustrial to 2000', preindustrial, year_2000),
    ('2000 to present', year_2000, present),
    ('Preindustrial to present', preindustrial, present),
]:
    print(f"{label}: pH decline = {float(earlier['pH'] - later['pH']):.3f}")
```

Preindustrial: pH = 8.142
2000: pH = 8.042
Present: pH = 7.982
Preindustrial to 2000: pH decline = 0.100
2000 to present: pH decline = 0.060
Preindustrial to present: pH decline = 0.160

Instructor answer

Preindustrial pH is **8.142**, pH in 2000 is **8.042**, and present pH is **7.982**. The declines are **0.100** (preindustrial → 2000), **0.060** (2000 → present), and **0.160 pH units** in total. All comparisons hold TA, temperature and salinity fixed. These are equilibrium differences for the exercise's CO₂ values, not an attribution to any individual's emissions.

f) Aragonite and calcite saturation

Question — part f

At the exercise's present-day CO₂ value of **430 ppm**, calculate the saturation states of **aragonite** and **calcite**, two forms of CaCO₃ used by marine calcifiers. Interpret the values relative to a saturation state of one.

```
# Your calculation
```

```
target_aragonite = float(present['saturation_aragonite'])  
print(f"Present Omega_aragonite = {target_aragonite:.3f}")  
print(f"Present Omega_calcite = {float(present['saturation_calcite']):.3f}")
```

```
Present Omega_aragonite = 1.975  
Present Omega_calcite = 3.075
```

Instructor answer

. and .. Saturation state is $\Omega = [Ca^{2+}][CO_3^{2-}]/K_{sp}$ for the specified mineral. Both values exceed one, so the water is supersaturated with respect to both minerals. Below one it is undersaturated. Supersaturation describes the chemical driving force; it does not alone establish a calcification rate or guarantee healthy organisms. Aragonite has the lower saturation state in this calculation.

g) The 2100 high-CO₂ endpoint and warming

Question — part g

Using the exercise's RCP8.5-labelled **935 ppm** CO₂ endpoint for 2100, calculate **pH, DIC and aragonite saturation** at 15 °C and then with **3 °C warming (18 °C)**. Keep TA = 2100 µmol/kg and salinity = 35. Compare both endpoints with the present and discuss the results.

```
# Your calculation
```

```

future = equilibrium(935.0)
future_warm = equilibrium(935.0, temperature=18.0)
for label, state in [('Present, 15 degC', present),
                    ('2100, 15 degC', future), ('2100, 18 degC', future_warm)]:
    show_state(label, state)
    print(f"  pH change from present: {float(state['pH'] - present['pH']):+.3f}")

```

Present, 15 degC: pH = 7.982; DIC = 1918.7 $\mu\text{mol/kg}$; TA = 2100.0 $\mu\text{mol/kg}$; Omega_aragonite = 1.975; Omega_calcite = 3.075
 pH change from present: +0.000
 2100, 15 degC: pH = 7.680; DIC = 2027.8 $\mu\text{mol/kg}$; TA = 2100.0 $\mu\text{mol/kg}$; Omega_aragonite = 1.068; Omega_calcite = 1.663
 pH change from present: -0.302
 2100, 18 degC: pH = 7.685; DIC = 2012.5 $\mu\text{mol/kg}$; TA = 2100.0 $\mu\text{mol/kg}$; Omega_aragonite = 1.210; Omega_calcite = 1.871
 pH change from present: -0.297

Instructor answer

Case	pH	DIC ($\mu\text{mol/kg}$)	Aragonite saturation
Present: 430 ppm, 15 °C	7.982	1918.7	1.975
2100: 935 ppm, 15 °C	7.680	2027.8	1.068
2100: 935 ppm, 18 °C	7.685	2012.5	1.210

Higher CO_2 increases DIC, lowers pH by about **0.302**, and reduces carbonate availability and aragonite saturation. At fixed 935 ppm, warming by 3 °C changes pH by only **+0.005**, but decreases DIC by **15.3 $\mu\text{mol/kg}$** and increases aragonite saturation by **0.142** (about 13%). It therefore has a small direct pH effect here, but a noticeable saturation effect. Both future states are still supersaturated with respect to aragonite, although less so than the present.

The endpoint is prescribed, not a new forecast or a transient model result. The small pH sensitivity does not make warming harmless: heat stress can cause coral bleaching, which this chemistry calculation does not model ([NOAA: coral bleaching](#)).

h) Alkalinity needed to restore aragonite saturation

Question — part h

At **935 ppm** CO_2 , calculate the total alkalinity required to restore the **present-day aragonite saturation from (f)**. Report both the required TA and its increase above 2100 $\mu\text{mol/kg}$. Use 15 °C for the main answer and also report the 18 °C alternative from (g). Does restoring saturation restore present pH? Explain what this calculation can and cannot establish about alkalinity addition as a way to protect coral reefs.

Your calculation

```

oea_states = {}
for temperature in (15.0, 18.0):
    state = pyco2.sys(
        par1=target_aragonite, par1_type=11, # prescribed Omega_aragonite
        par2=935.0, par2_type=9, # prescribed dry-air xCO2
        temperature=temperature, salinity=SALINITY,
        pressure=PRESSURE_DBAR, **config.chemistry,
    )
    oea_states[temperature] = state
    show_state(f'Restored saturation at {temperature:g} degC', state)

```

```

print(f" Required TA enhancement: {float(state['alkalinity']) - INITIAL_TA:.1f} umol/kg")
print(f" pH change from present: {float(state['pH']) - present['pH']:.3f}")

# Independently re-solve with inferred TA + prescribed CO2 to check the inversion.
check = pyco2.sys(
    par1=float(state['alkalinity']), par1_type=1,
    par2=935.0, par2_type=9,
    temperature=temperature, salinity=SALINITY,
    pressure=PRESSURE_DBAR, **config.chemistry,
)
assert abs(float(check['saturation_aragonite']) - target_aragonite) < 1e-8
assert abs(float(check['pH']) - state['pH']) < 1e-8

```

Restored saturation at 15 degC: pH = 7.813; DIC = 2779.1 umol/kg; TA = 2922.4 umol/kg; Omega_aragonite = 1.975;
 Omega_calcite = 3.075
 Required TA enhancement: 822.4 umol/kg
 pH change from present: -0.169

Restored saturation at 18 degC: pH = 7.791; DIC = 2590.2 umol/kg; TA = 2737.2 umol/kg; Omega_aragonite = 1.975;
 Omega_calcite = 3.054
 Required TA enhancement: 637.2 umol/kg
 pH change from present: -0.191

Instructor answer

Temperature	Required TA (μmol/kg)	Added TA (μmol/kg)	pH	DIC (μmol/kg)
15 °C	2922.4	822.4	7.813	2779.1
18 °C	2737.2	637.2	7.791	2590.2

Both states reproduce the target aragonite saturation of **1.975** by construction. For the main 15 °C answer, TA must rise by about **39%**. The warmer alternative requires less TA to attain the same saturation state. Neither restores present pH (7.982): saturation and pH are different properties, and restoring one does not fix the other.

The target saturation is **prescribed**, and TA is **inferred**. The final assertions check the chemistry inversion; they do not independently validate alkalinity enhancement as a reef intervention. DIC rises as the water equilibrates with the prescribed atmosphere. There is no closed ocean–atmosphere carbon inventory here, and this is a concentration increment, not a total dose or addition rate. A dose would require the treated water mass and a specified material; adding carbonate mineral also adds DIC, unlike an idealized pure-TA addition.

Higher equilibrium DIC is consistent with potential CO₂ uptake, but these states do not quantify uptake rates, net atmospheric removal, mixing, precipitation, cost or ecological feasibility. Restored saturation also does not remove thermal stress or guarantee coral survival. A coupled process model and site-specific evidence would be needed to assess an intervention.

Question — final check

End of exercise 9. Check that every answer states its conditions and units, and distinguishes prescribed inputs from calculated outputs.